System Analysis & Design

Architecture, Modeling & Technical Specification

Podcast Streaming Platform

Graduation Project - Faculty of Computers and Information
ASP.NET Core Web API | React / Next.js | PostgreSQL | Agile Scrum

Prepared by: Project Development Team

Academic Year 2025 - 2026

Table of Contents

1. Problem Statement & Objectives
2. Software Architecture
    - 2.1 Architecture Diagram
3. Database Design
    - 3.1 ERD
    - 3.2 Schema
4. Data Flow & Behavior
    - 4.1 DFD (L0 + L1)
    - 4.2 Sequence Diagram
    - 4.3 Activity Diagram
    - 4.4 State Diagram
    - 4.5 Class Diagram
5. UI/UX Design
    - 5.1 Wireframes
    - 5.2 Design Guidelines
6. Deployment & Integration
    - 6.1 Tech Stack
    - 6.2 Deployment Diagram
    - 6.3 Component Diagram
7. API & Testing
    - 7.1 API Endpoints
    - 7.2 Testing Strategy
    - 7.3 Deployment Strategy

1. Problem Statement & Objectives

Podcast consumers currently rely on YouTube, which causes excessive battery drain, intrusive ads, and lacks audio-specific management tools. Content creators have no streamlined way to extract, trim, and redistribute audio segments as standalone podcast episodes. The Podcast Streaming Platform addresses this by providing a web-based system for discovering, streaming, and organizing podcast content extracted from YouTube using yt-dlp and FFmpeg. It supports three roles: Listeners, Creators, and Administrators.

ID	Objective
O1	Web app for podcast discovery, streaming, and content management from YouTube sources
O2	Automated YouTube-to-audio conversion with precise timestamp trimming (yt-dlp + FFmpeg)
O3	Creator tools for metadata management, categorization, and engagement analytics
O4	High performance (< 2s load, < 500ms API), security (JWT, OWASP), and scalability (500+ users)
O5	Complete, tested, documented system delivered via Agile Scrum within 16 weeks

Table 1.1: Project objectives.

2. Software Architecture

The platform uses a Layered Monolith architecture with internal service modules. This balances simplicity (suitable for 4-person team, 16-week timeline) with extensibility (modules can be extracted to microservices later). Dependencies flow downward only through well-defined interfaces.

Layer	Technology	Responsibility
Presentation	React 18 / Next.js 14 + Tailwind CSS	UI rendering, state management, client-side routing
API Layer	ASP.NET Core 8 Web API + JWT middleware	RESTful endpoints, auth, validation, rate limiting
Service Layer	C# business logic with DI	Domain services, audio orchestration, caching
Infrastructure	EF Core, Redis, yt-dlp/FFmpeg wrappers	DB access, external services, file storage
Data Layer	PostgreSQL 16 + Redis + Blob Storage	Persistent data, sessions, audio files

Table 2.1: Architecture layers.

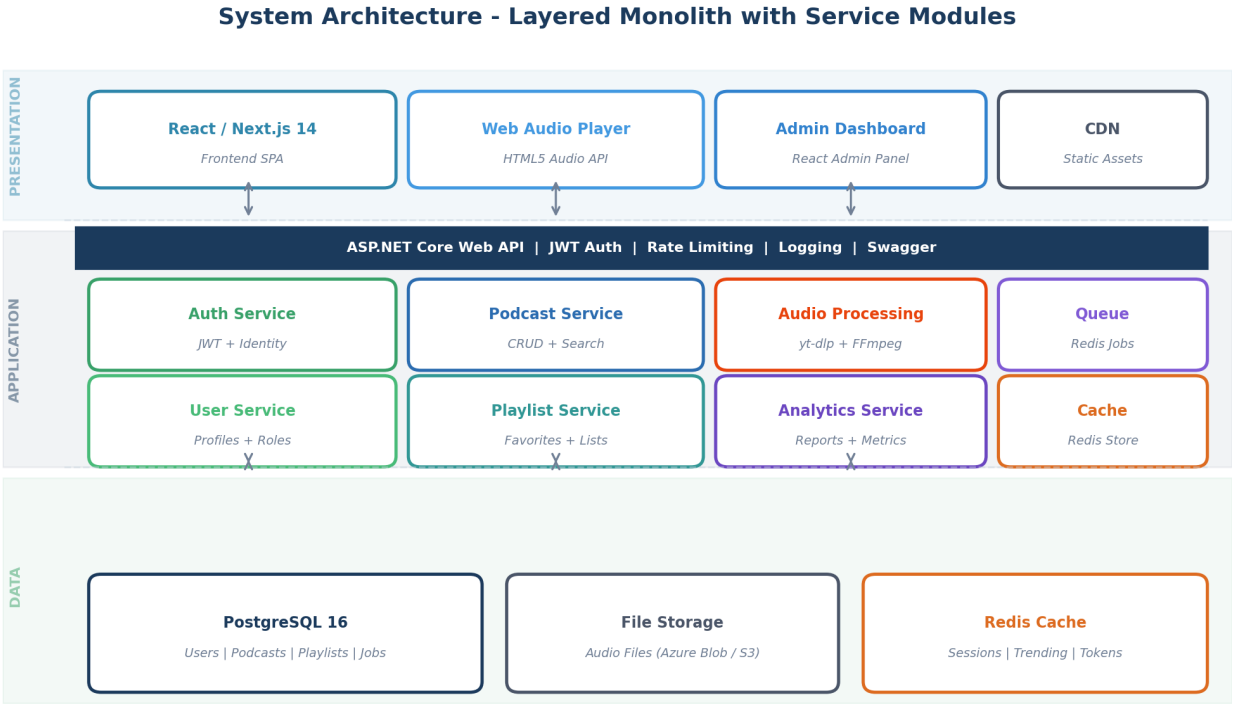

Figure 2.1: System architecture with 3 tiers and 8 service modules.

3. Database Design & Data Modeling

The schema follows Third Normal Form (3NF) with 8 core entities. PostgreSQL 16 provides full-text search via GIN indexes on Podcasts.Title and Description.

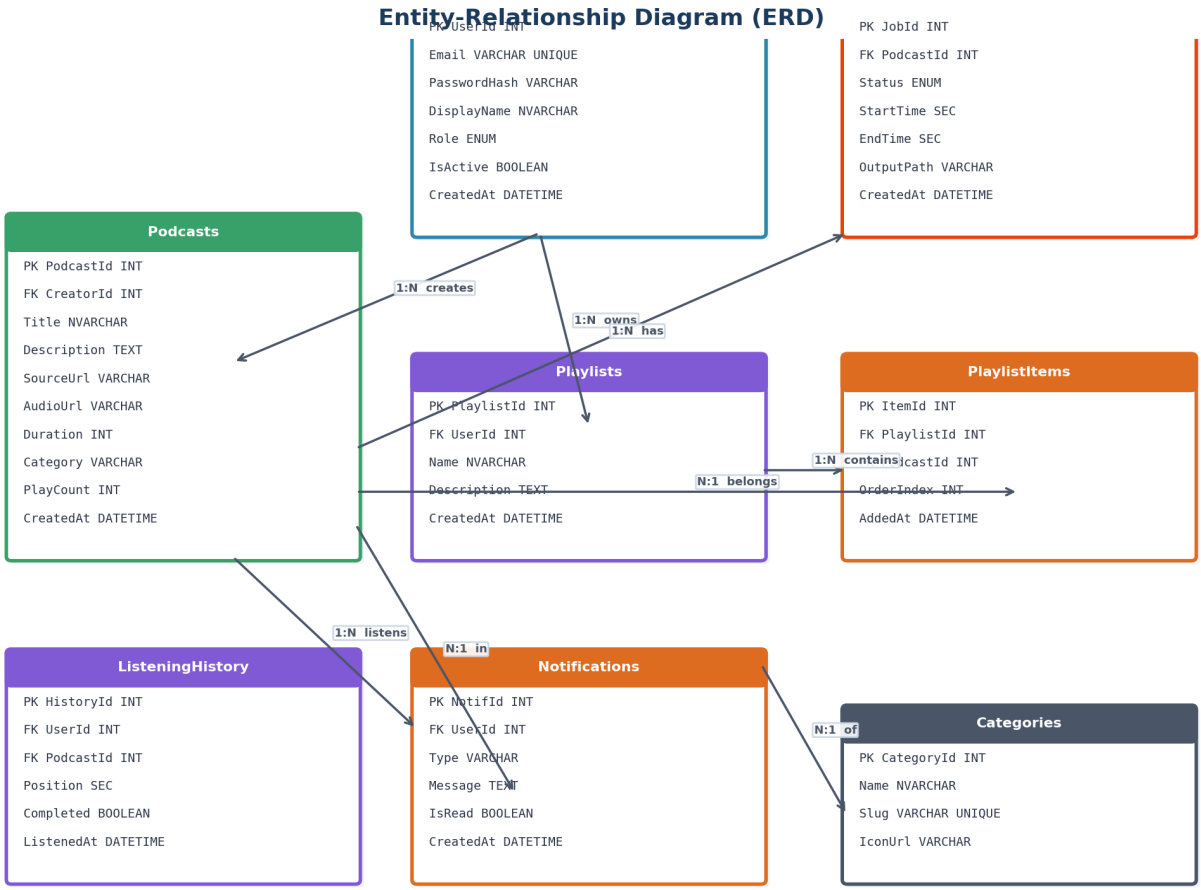

Figure 3.1: Entity-Relationship Diagram with all 8 entities.

Table	PK	Attributes
Users	UserId (PK)	Email, PasswordHash, DisplayName, Role, IsActive, AvatarUrl, CreatedAt
Podcasts	PodcastId (PK)	CreatorId (FK), Title, Description, SourceUrl, AudioUrl, Duration, Category, PlayCount
Playlists	PlaylistId (PK)	UserId (FK), Name, Description, CreatedAt
PlaylistItems	ItemId (PK)	PlaylistId (FK), PodcastId (FK), OrderIndex, AddedAt
AudioJobs	JobId (PK)	PodcastId (FK), Status, StartTime, EndTime, OutputPath, CreatedAt
Categories	CategoryId (PK)	Name, Slug (UNIQUE), IconUrl
ListeningHistory	HistoryId (PK)	UserId (FK), PodcastId (FK), Position, Completed, ListenedAt
Notifications	NotifId (PK)	UserId (FK), Type, Message, IsRead, CreatedAt

Table 3.1: Complete database schema (3NF).

4. Data Flow & System Behavior

4.1 Data Flow Diagrams

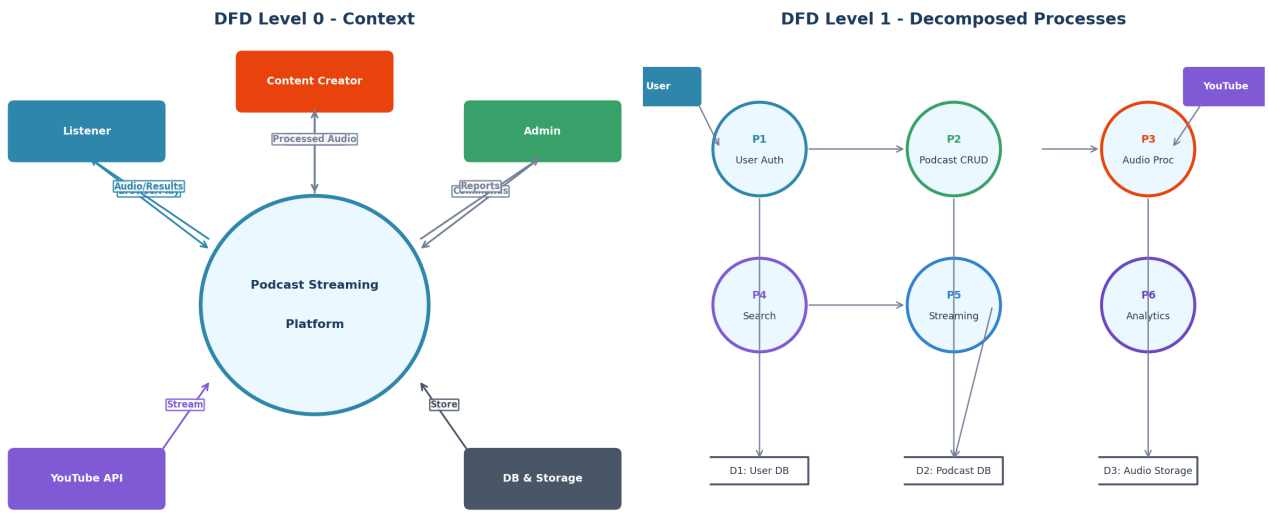

Figure 4.1: DFD Level 0 (left) and Level 1 (right) showing data flows.

4.2 Sequence Diagram

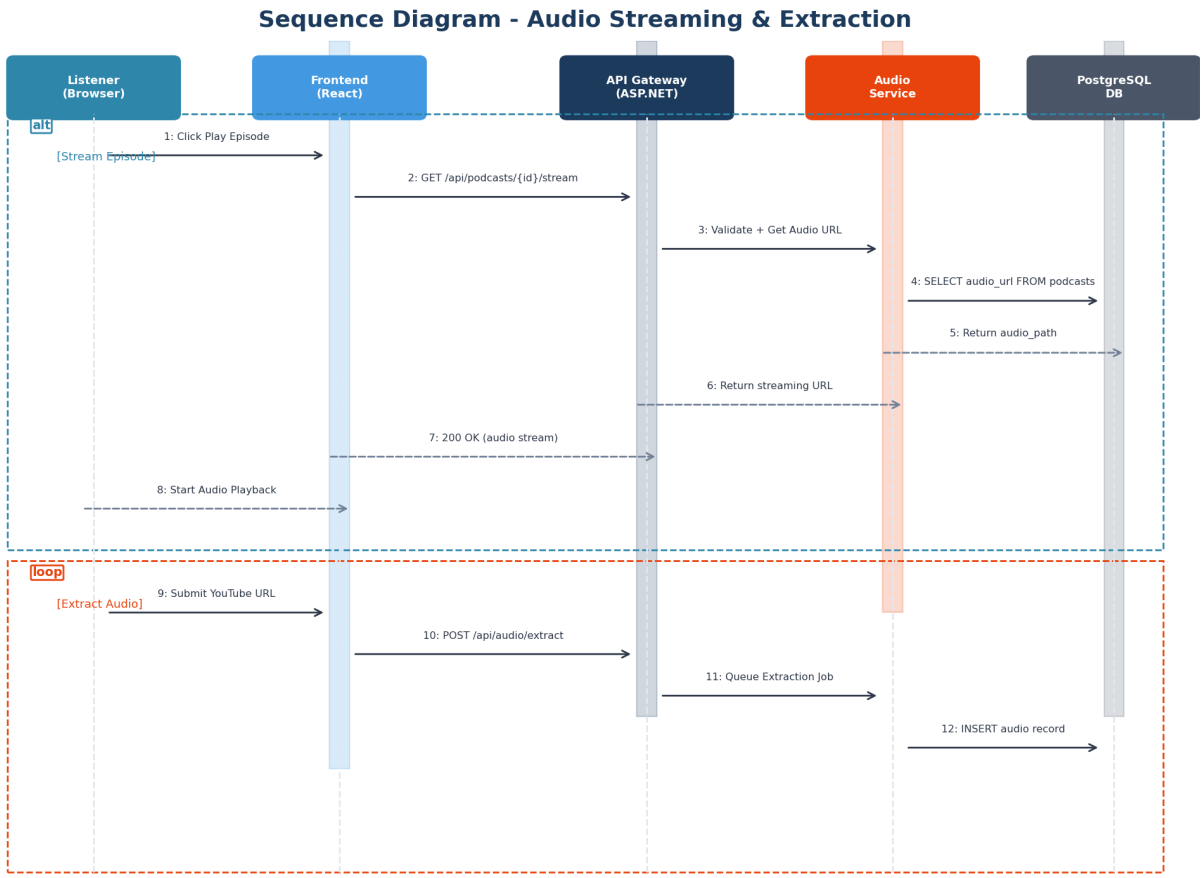

Figure 4.2: Sequence diagram for audio streaming and extraction.

4.3 Activity Diagram

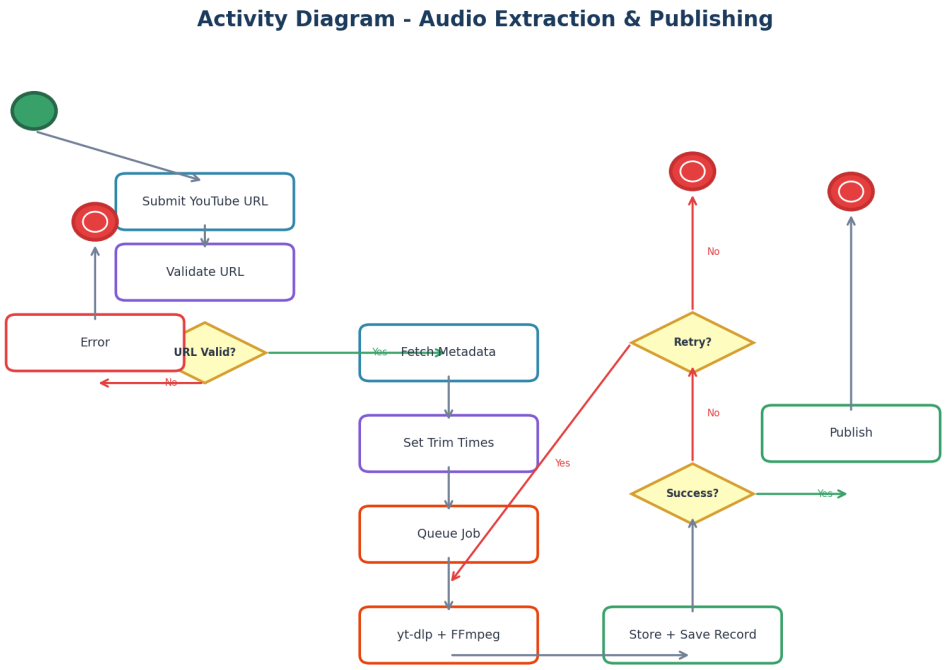

Figure 4.3: Activity diagram for audio extraction and publishing workflow.

4.4 State Diagram

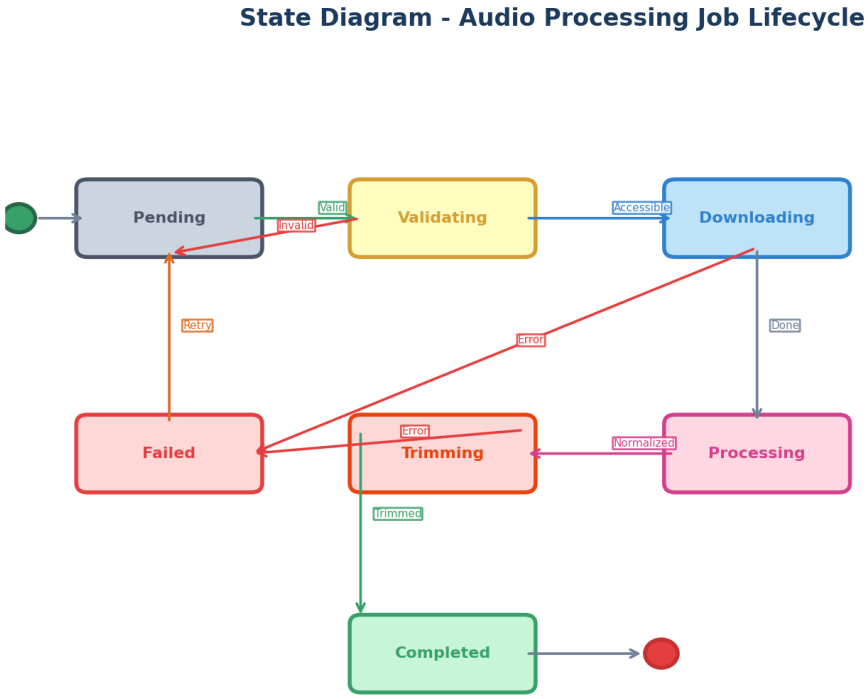

Figure 4.4: State diagram for audio processing job lifecycle (7 states, retry loop).

4.5 Class Diagram

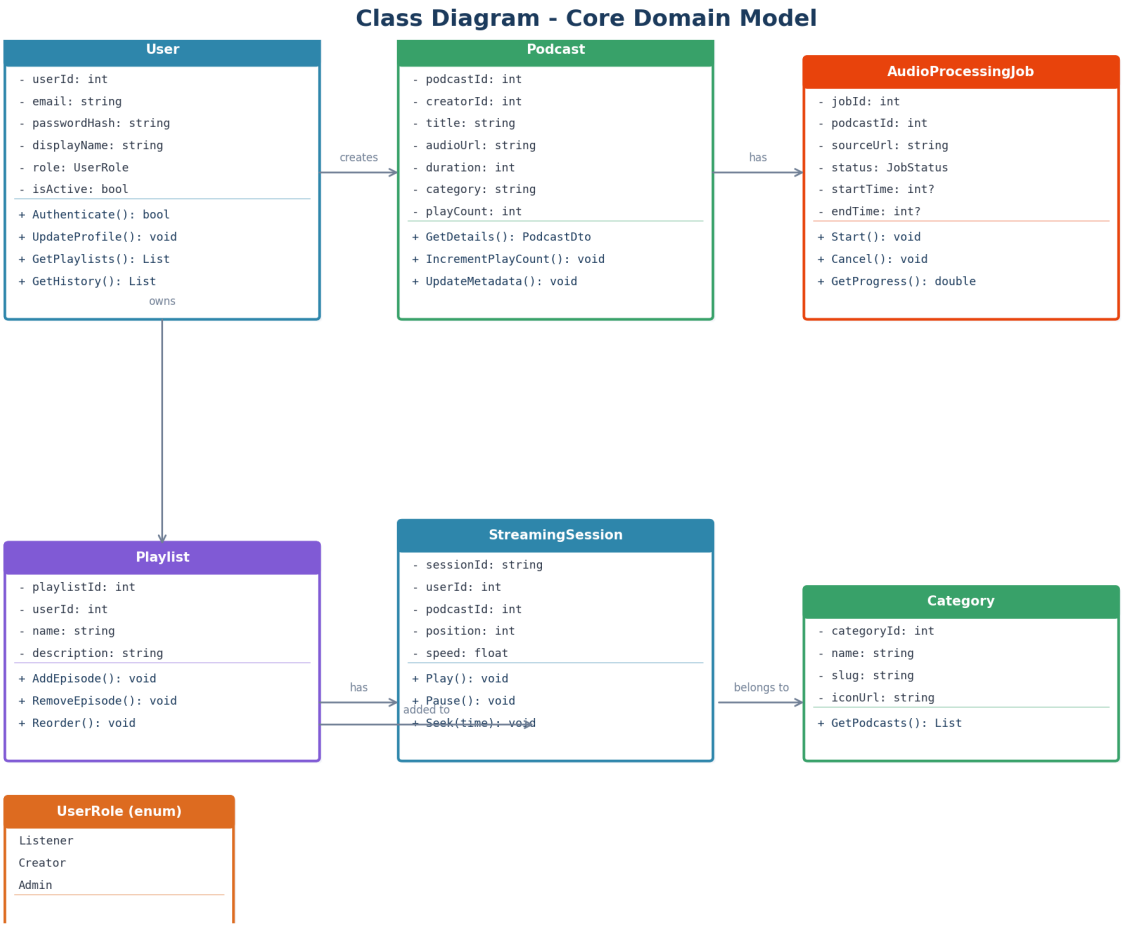

Figure 4.5: Class diagram with 6 core classes and relationships.

5. UI/UX Design & Prototyping

5.1 Wireframes

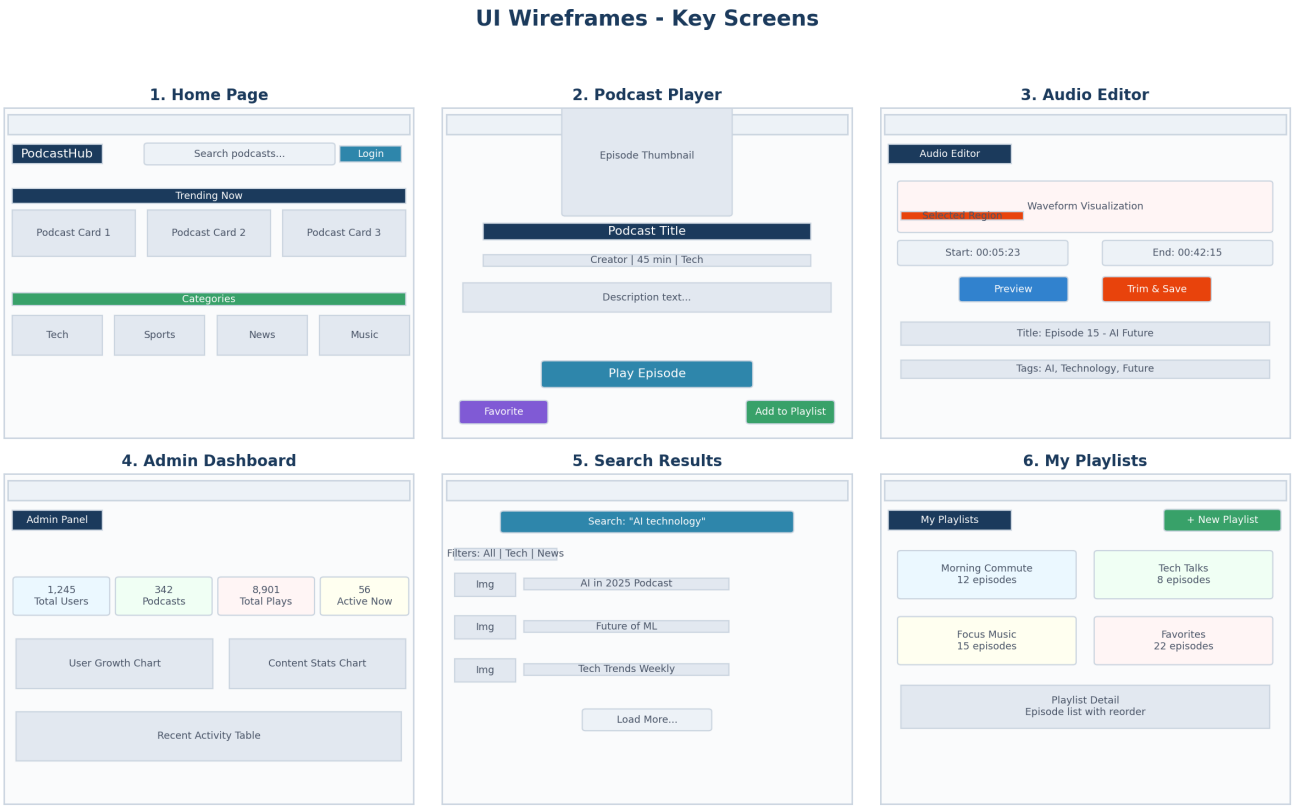

Figure 5.1: Wireframes for 6 key screens.

5.2 Design Guidelines

Aspect	Guideline
Colors	Primary #1B3A5C, Secondary #2E86AB, Accent #E8430C, Success #38A169, Error #E53E3E, BG #F0F4F8
Typography	Headings: Inter Bold 18-28px, Body: Inter Regular 14-16px, Mono: timestamps/technical
Spacing	8px grid, 16px padding, 24px margins, 8px border-radius (cards), 6px (buttons)
Responsive	Mobile-first; breakpoints 640/768/1024/1280px; single-column below 768px
Accessibility	WCAG 2.1 AA; 4.5:1 contrast; full keyboard nav; ARIA labels; alt text

Table 5.1: UI/UX design guidelines.

6. System Deployment & Integration

6.1 Technology Stack

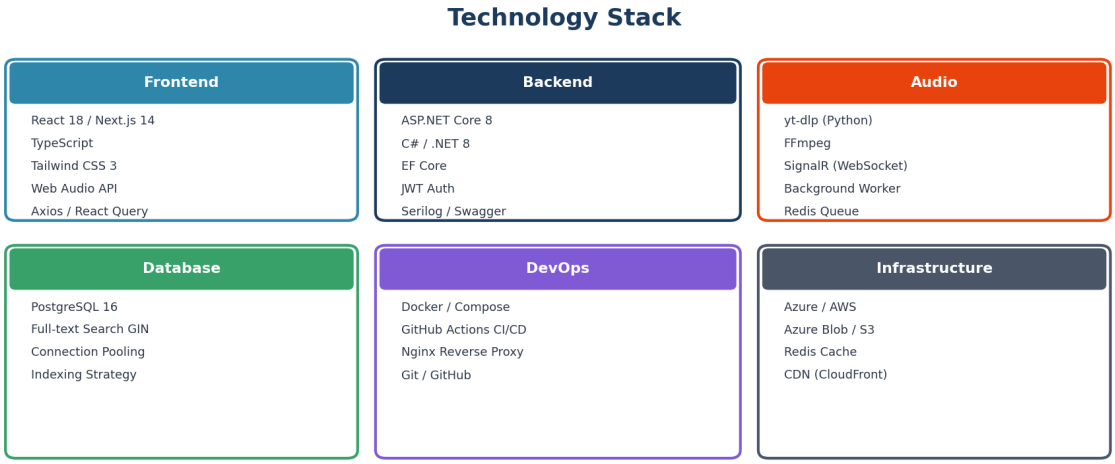

Figure 6.1: Technology stack by tier.

6.2 Deployment Diagram

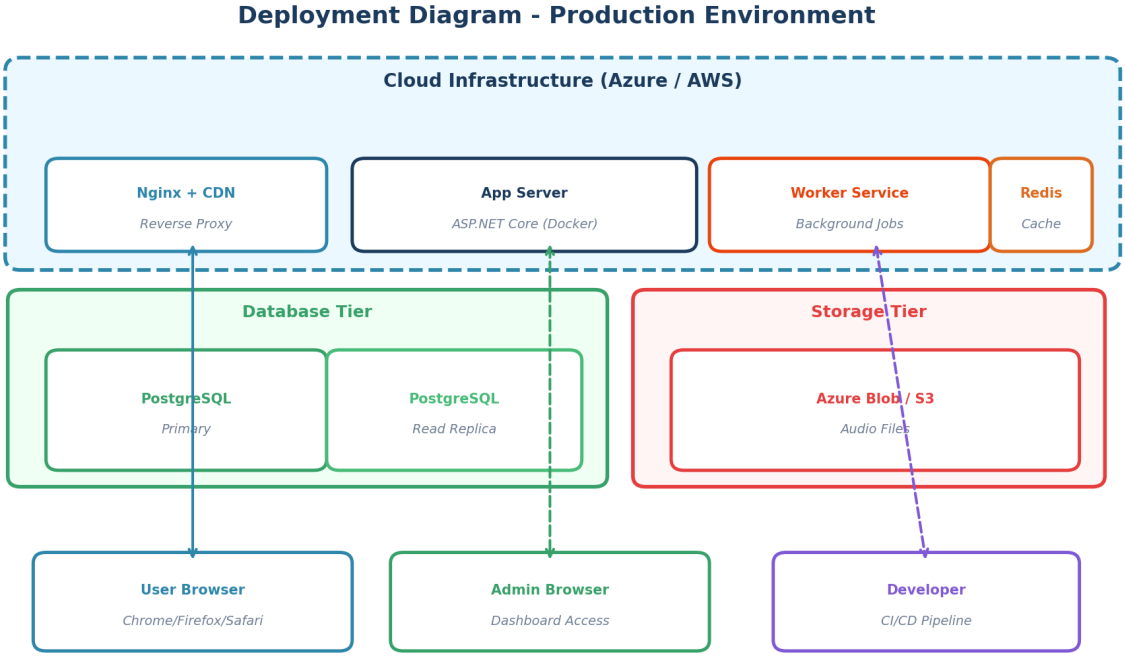

Figure 6.2: Production deployment topology.

6.3 Component Diagram

Component Diagram

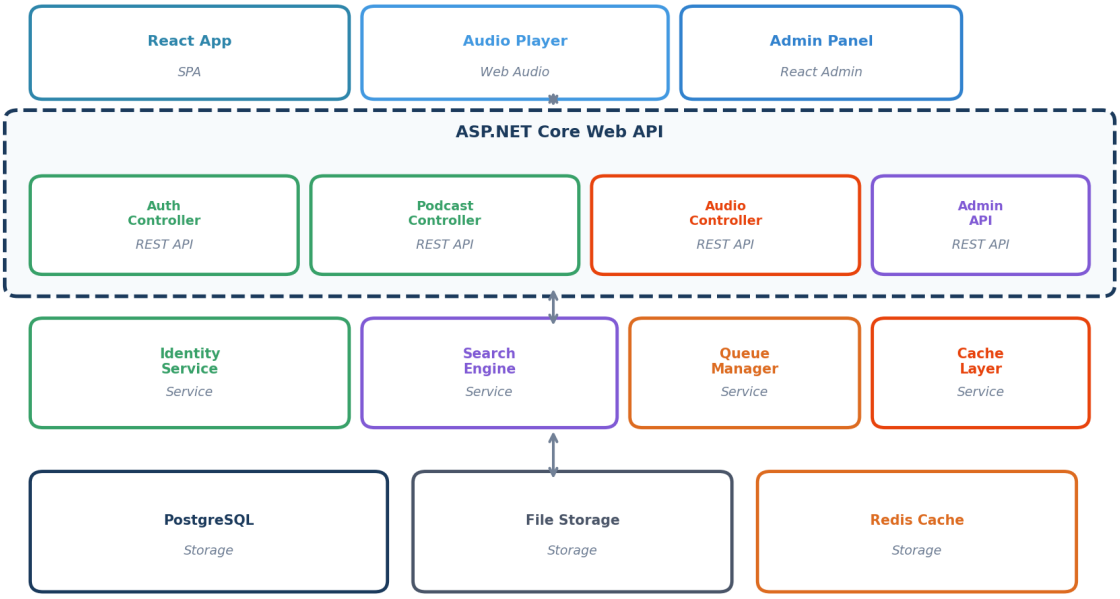

Figure 6.3: System components across 3 tiers.

7. API Documentation & Testing

7.1 API Endpoints

Method	Endpoint	Description	Access
POST	/api/v1/auth/register	Register user	Public
POST	/api/v1/auth/login	Authenticate	Public
GET	/api/v1/podcasts	List (paginated)	Public
POST	/api/v1/podcasts	Create from URL	Creator
POST	/api/v1/audio/extract	Submit extraction	Creator
POST	/api/v1/audio/{id}/trim	Trim audio	Creator
GET	/api/v1/search?q={}	Full-text search	Public
GET	/api/v1/playlists	List playlists	Auth
POST	/api/v1/playlists	Create playlist	Auth
GET	/api/v1/admin/analytics	Platform metrics	Admin

Table 7.1: RESTful API endpoints.

7.2 Testing Strategy

Type	Tools	Scope	Target	Freq
Unit Tests	xUnit + Moq	Business logic, services	60%+ coverage	Every commit
Integration	xUnit + TestServer	API endpoints, DB	Critical paths	Per sprint
E2E	Playwright	Key user journeys	5-10 scenarios	Per milestone
Performance	k6	Load (500 users)	API + streaming	Pre-release
UAT	Manual	Stakeholder validation	All Must-Have	Pre-submission

Table 7.2: Testing strategy.

7.3 Deployment Strategy

Stage	Env	Trigger	Actions
Build	CI	Push to branch	Restore, compile, unit tests
Test	Staging	PR to main	Integration + E2E
Staging	Staging	Merge develop	Docker deploy, smoke tests
Production	Prod	Tag release	Blue-green deploy, health check

Table 7.3: CI/CD pipeline.

Version	Date	Author	Changes
1.0	2026-04-30	Project Team	Initial improved release

Table 7.4: Revision history.